AI Engineering Portfolio — Updated Evidence-Backed Draft

NEMSU Tandag Campus · SIMS · School Year 2026–2027

Status: Draft; screenshot and first-person review required

Version 7 update — November 10, 2026

The independent Version 7 archive review found **41 Git commits**, **93 test methods**, and **17 JPG development screenshots**. These facts describe the archived repository, not a guaranteed current local workstation state. The project’s Windows/Laragon testing record reports **93 passing tests with 633 assertions**, but this external audit did **not** reproduce the complete PHPUnit run. It verified PHP syntax, SQLite integrity and foreign keys, and a schema-only SQL import.

Evidence still needing human review

? [ ] Match each prompt case study below to the authentic VS Code/ChatGPT conversation capture.

? [ ] Identify the actual model/version only if recoverable from settings or a saved session record.

? [ ] Insert genuine screenshots from the `screenshots/` folder with accurate captions, after checking what each image shows.

? [ ] Write the student's first-person evaluation for each chosen prompt and correction; do not generate invented autobiographical responses.

? [ ] Attach a current local terminal screenshot of `php artisan test`, `npm run build`, and any remaining formatting findings.

? [ ] Review descriptions of roles/permissions against Version 7, including Administrator-only Student API writes and last-Administrator safeguards.

? [ ] Update file names and broken cross-references, then export the complete file as **`AI_PORTFOLIO.pdf`**.

Integrity note: The proposed specialized agents in `AGENT_DESIGN_UPDATED.md` are not evidence that multiple independent agents actually executed. The existing portfolio documents a sequential assistant-led workflow.

This draft records only prompts and outcomes present in the supplied project

conversation and current repository work. It is not a substitute for the

rubric's requested portfolio PDF and screenshots. The student should verify each entry against their full chat history, add screenshots, and write their own evaluation before submission.

Tools and model record

? Development environment: Copilot SDK in VS Code.

? Model/version: not recorded in the conversation evidence available for this draft.

? Purpose: inspect the existing Laravel workspace, make code changes, execute application commands/tests, and update project documentation.

? Observed strengths in this workflow: workspace file inspection, focused patches, route/test execution, and iterative corrections in one environment.

? Observed limitation: the session record does not identify a model version or provide token-cost telemetry.

? Independent specialized agents: none were used for the work recorded here.

? Other AI platforms tested for comparison: none documented. A comparative study would be dishonest without genuine side-by-side runs.

? PDF deliverable: this Markdown file is the source draft. It still needs student review, genuine screenshots, and export as `AI_PORTFOLIO.pdf`.

Prompt 1 — debugging PHP namespace placement

Objective: Resolve a test-file fatal error preventing Laravel tests from starting.

Original prompt excerpt (user):

Fatal error: Namespace declaration statement has to be the very first statement or after any declare call in the script in

C:\laragon\www\student-information-system\tests\Feature\SubjectCrudTest.php

on line 4

Problem identified: PHP does not allow bytes or output before the opening PHP tag when a namespace declaration follows.

AI result / correction recorded: The leading blank line before `<?php` was removed in the affected test file. The same issue was checked in other touched PHP files.

Verification evidence: The Subject CRUD test later passed, and route loading was verified during local-run troubleshooting. Preserve the actual terminal screenshot or test output from the student's environment if it is required as submission evidence.

Student evaluation: Add a first-person assessment of whether this fix addressed the reported failure and what was checked before changing the file.

Prompt 2 — adding management navigation

Objective: Make the existing management module accessible from the Breeze navigation.

Original prompt excerpts (user):

move the manage students to nav bar

`<x-nav-link :href="route('enrollments.index')"

:active="request()->routeIs('enrollments.*')">Enrollments</x-nav-link>`

`<x-responsive-nav-link :href="route('enrollments.index')"

:active="request()->routeIs('enrollments.*')">Enrollments</x-responsive-nav-link>`

AI result recorded: Management links were added to Breeze desktop and

responsive navigation, preserving route-based active states.

Verification evidence: Navigation changes were made in the existing Breeze layout; prior project work reports Blade and related feature checks. Add screenshots showing desktop and mobile navigation if required.

Student evaluation: Describe whether the placement and responsive behavior matched the intended navigation and any manual browser checks.

Prompt 3 — refining the scope to the existing project

Original prompt excerpt (user):

This is an existing project, NOT a new project.

DO NOT recreate Laravel, overwrite working modules unnecessarily, replace Breeze authentication, reset the database, or destroy existing data.

Refinement: The prompt then supplied detailed Laravel, authorization, CRUD, database, API, test, and documentation requirements and directed the work to inspect before implementing. This bounded changes to the existing architecture and data.

Evaluation: Existing routes, migrations, role rules, and tests were inspected before adding the Student API. No new Laravel application was created, and only the additive Sanctum token migration was applied.

Prompt 4 — implementing and validating the required Student API

Objective: Implement all five required REST endpoints without replacing existing authentication or schema.

Original prompt excerpts (user):

THIS IS A HIGH-PRIORITY REQUIREMENT.

Provide tests for all five mandatory endpoints.

Use the existing Student database schema.

AI result: Added Sanctum bearer-token authentication, five Student API routes, an API controller and resource, shared student validation, and admin-only API writes. Kept the web CRUD controller and database schema.

Problem found during validation: The API resource initially reused the web route names. Route inspection and a web CRUD test exposed that `students.index` resolved to the API route. API route names were prefixed `api.` so web route generation remains unchanged. Tests also showed there is no Student factory; test data was changed to explicit model creation rather than adding an unnecessary factory.

Verification result at that development stage: `StudentApiTest`, `StudentCrudTest`, and `RegistrationTest` passed in the then-current full run (56 tests / 280 assertions). Later catalog work expanded the test suite; see Prompt 6 for final verification totals.

Student evaluation: Review the authorization policy, payload validation, and response codes; document any changes you personally made or rejected.

Prompt 5 — validation prompt and iterative failure correction

Actual validation prompt excerpt (user):

Run:

>

`php artisan test`

>

Report:

>

- Tests executed

- Tests passed

- Tests failed

- Actual error messages

- Fixes applied

>

Do not claim that tests passed unless they actually ran successfully.

AI-generated output: Added a partial-update test for the Student API and used `sometimes|required` as one string inside an array-based Laravel rule set.

Problem found: The full test run reported: `Method Illuminate\Validation\Validator::validateSometimes|required does not exist.` The partial-update request returned HTTP 500 instead of the expected 200.

Correction applied: Changed the partial required rule to the two Laravel rule entries `['sometimes', 'required']`, while preserving the existing full PUT rules.

Result validated at that stage: The targeted Student API and CRUD tests passed after the correction. The complete suite then passed with 56 tests and 280 assertions; this historical count was superseded by the latest result below.

A separate subsequent test run briefly reported `Undefined variable $existing` because an email-uniqueness assertion was placed in the student-update test instead of the student-create test. The assertion was moved into the test

scope where `$existing` is defined; the targeted CRUD suites and final full suite then passed.

The schema-export smoke test also caught an initial DDL ordering problem (`no such table: main.enrollments` while importing an index before its table). The export query was changed to emit table definitions before indexes and excluded SQLite's internal `sqlite_sequence`; the resulting schema-only SQL successfully imported into an in-memory SQLite database.

Prompt 6 — integrating the supplied academic catalog

Objective (carried-forward task summary; verbatim original prompt is not available here): Add the supplied NEMSU Tandag Campus 2026–2027 college, department, and program hierarchy to the existing SIMS without creating subjects from degree programs, changing the existing enrollment relationship, or replacing legacy student course data.

Implementation result: Added normalized College → Department → Program tables/models and an idempotent, non-destructive seeder; connected dependent student form selectors and server-side relationship validation; retained the Student `course` string and Student → Enrollment → Subject design; added an authenticated searchable directory and dashboard catalog counts.

Problem identified during review: Simply making `course` a catalog-only choice would reject older student values on update. Passing selector IDs through mass-assignment was also unnecessary because only the selected program name belongs in the existing Student schema.

Correction applied: Existing unchanged legacy course values remain valid on updates; web and API controllers persist only the Student's existing fillable fields. The seeder does not overwrite a college if an abbreviation

conflict is found; it reports the conflict rather than silently changing
catalog data.

Verification result: The focused catalog, Student CRUD, Student API, and dashboard tests passed (16 tests, 149 assertions). The full suite then passed 62 tests / 324 assertions. The local additive migrations and seeder completed; verification showed 7 colleges, 13 departments, 22 programs, and unchanged counts of one student, one subject, and one enrollment. The production frontend build, Pint on changed PHP files, Composer validation, and schema-only export import also passed. The schema import contained no catalog rows.

Student evaluation: Check the supplied catalog against its original source and describe whether keeping a string-valued Student `course` field is an appropriate compatibility choice for the final project.

Prompt 7 — upgrading the administration dashboard and role management

Original prompt evidence: The user supplied a detailed Administration upgrade brief in this conversation. Its explicit requirements included:

Transform it into a useful system management dashboard.

Only Administrators can change account roles.

Prevent: Unauthorized privilege escalation; Users changing their own role through profile requests; Accidental self-demotion by an Administrator; Demoting the last remaining Administrator.

Never fabricate historical activity.

Inspection findings: The prior Administration page was a static welcome view. `/admin` already required `role:admin` and was linked by the existing

dashboard and navigation. Registration already assigned `staff`, and `ProfileUpdateRequest` accepted only name/email while `User::$fillable` excluded role. Breeze profile/account flows, Student/Subject/Enrollment CRUD, the API, and the academic catalog were preserved.

Implementation result: Replaced the static page with an invokable admin dashboard controller and live totals; added summaries, management shortcuts, catalog/program details, recent records, and data-backed program/year/status reports. Added an administrator-only searchable and filterable user directory, safe account detail pages, and role-only updates. No admin account deletion, suspension, or historical audit log was added. Recent records are labeled by their creation timestamp, not presented as action history.

Security review and correction: The user-edit screen is not the security boundary: the role update is restricted by authenticated admin middleware and server-side validation. A role-only endpoint avoids mass-assigning user attributes; self-role changes are rejected, and the update is transactional with a post-change check that rolls back any change leaving no administrator. The existing profile update already ignored a crafted `role` field, and a regression test now verifies this.

Verification evidence at that stage: The new dashboard/user-management tests cover database totals, reports, catalog counts, route access for staff/guests, search/filtering, role validation, promotion/demotion of another account, self-demotion protection, and profile role tampering. The complete PHPUnit suite then passed **70 tests / 374 assertions**; `npm run build` passed; Pint passed for the changed admin PHP files. No migrations or destructive DB commands were run for the Administration changes.

Workflow note: This was completed in the current assistant session. No

independent specialist agent handoffs or screenshots are claimed.

Token and context management — 5-point rubric evidence

The actual workflow used progressively broader context rather than copying the

entire repository into each decision. Examples below describe actions visible

in the development session; no token totals or model-specific token prices

were recorded.

| Technique | Genuine example from this work | Quality check |

|---|---|---|

| Single-file context | Inspected `StudentDataRequest.php` separately when reviewing the shared validation behavior. | Compared its rules against the student web/API tests and route binding. |

| Feature-based context | Read Student CRUD and Student API tests together while filling student validation, auth, and not-found gaps. | Ran the relevant test filter before the full suite. |

| Module-based context | Reviewed Subject, Enrollment, Dashboard, RoleAccess, and Breeze authentication tests as separate feature modules. | Added only module-relevant assertions and preserved existing role behavior. |

| Smaller task prompts | The actual requests “put save bbutton in enrollments” and “move the manage students to nav bar” addressed bounded UI changes; the current request separately specified validation, docs, and portfolio requirements. | Confirmed the corresponding Enrollment/navigation tests and later ran the full suite. |

| Task decomposition | Work proceeded through inspect, focused implementation, targeted tests, documentation, and full verification rather than a single broad code-generation step. | Each stage had observable tool output; the current test additions pass their targeted run. |

| Incremental generation | API implementation was followed by tests, then route naming was adjusted after a regression surfaced. | The route list and regression test confirmed web/API route names no longer collide. |

| Context reuse | A carried-forward conversation summary retained project constraints, prior decisions, and test results after earlier conversation turns were summarized. | The summary was checked against current files and commands before relying on it. |

| Agent specialization | Specialist responsibilities are documented as a proposed workflow. | No independent agents or specialist handoffs are claimed as actual work. |

Handbook questions

1. **How did unnecessary AI context get reduced?** By reading likely relevant files in bounded groups (for example, student request + Student tests), checking the specific migration and route list rather than re-reading the entire application, and using the carried-forward summary to retain prior decisions. This describes context organization, not a measured token reduction.
2. **How was prompt repetition avoided?** The summarized history was reused for already-set project constraints; subsequent work focused on the remaining tests and documentation. Actual earlier prompts are quoted above rather than reconstructed as new historical prompts.
3. **How were AI-generated outputs verified?** With route inspection, targeted feature tests, full PHPUnit runs, Pint, Composer validation, production asset build, migration status, and before/after record counts. The route-name collision, missing factory assumption, and formatting changes are recorded with their corrections below.
4. **How were token cost and output quality balanced?** Tests were first run for affected feature modules, then the complete suite was run; this gives fast feedback without treating narrow success as final proof. Existing controllers, schema, and middleware were reused rather than regenerated. No token cost telemetry was captured, so no numerical savings are claimed.

Critical verification examples

API route naming

? **AI-assisted implementation:** Added a resource API for the same Student model and resource names used by the web UI.

? **Problem found:** Laravel's shared route-name namespace caused

`students.index` to resolve to the API route, breaking the web test's redirect expectation.

? **Correction applied:** Put API resource routes under the `api.` name prefix, preserving the existing web route names.

? **Result validated:** `route:list` shows `api.students.*` and `students.*` independently; the web and API tests pass.

Test fixture assumption

? **AI-generated test setup:** Initially called `Student::factory()` even though this project has no Student factory.

? **Problem found:** The test runner reported `Call to undefined method App\Models\Student::factory()`.

? **Correction applied:** Used explicit Student records in tests instead of adding an unnecessary factory solely for these cases.

? **Result validated:** Student CRUD and API tests pass using the actual project setup.

Formatting and migration safety

? **AI-assisted changes:** Added API classes and a Sanctum personal-token migration.

? **Problem found:** Pint reported formatting differences; the token migration was pending and needed to be checked before modifying the local DB.

? **Correction applied:** Ran Pint on changed PHP files, reviewed the migration as additive, then applied it without a reset.

? **Result validated:** Pint passed; migration status showed the migration applied; student, subject, and enrollment counts remained one each.

Latest rubric-driven verification

The project’s Windows/Laragon testing record reports **93 tests / 633 assertions**. Composer validation, route listing, migration-status inspection, and the production asset build completed successfully. Repository-wide Pint reports formatting issues in existing files; after targeted formatting, Pint passes on the changed PHP files. Reported local results and the build warning are recorded in `docs/TESTING_AND_VERIFICATION.md`.

Evidence still to add

? Screenshots of the running dashboard, student/subject/enrollment pages, and API requests and responses.

? Exact model/version metadata and complete original prompt captures from the student's chat history.

? First-person evaluation of generated suggestions, corrections, and engineering trade-offs.

? A final PDF export after the student reviews and completes this draft.